

Architettura dei Calcolatori

Secondo Parziale – 18 maggio 2021 – 1h45

PARTE 1 – RISPOSTA SINGOLA - Ogni domanda ha una sola risposta VERA.

- Una risposta esatta fa acquisire il punteggio positivo riportato a fianco della domanda
 - Una risposta errata fa perdere il punteggio negativo riportato a fianco della domanda
 - Una risposta lasciata in bianco viene valutata 0
1. (2, -.5) Si supponga che il *program counter* (PC) contenga il valore 0x0040 0000. Qual è il salto in avanti più grande che può essere effettuato tramite l'istruzione jump and link (**jal**)? In altre parole, qual è il massimo incremento di indirizzo che il PC può avere dopo l'esecuzione della **jal**?
- a) Tra 0x400000 e 0x40FFFE
 - b) Tra 0x400000 e 0x400FFE
 - c) Tra 0x400000 e 0x407FFF
 - d) Nessuna delle precedenti
2. (2, -.5) Si consideri il seguente stralcio di codice assembly RISC-V

```
.globl main
.data
arr:   .word 10, 3, 4, 2, 1
n:     .word 5
.text
main:
    la    t0, arr
    la    t1, n
    lb    t1, 0(t1)
    lw    t2, 0(t0)

ciclo:
    addi   t1, t1, -1
    beq    t1, zero, exit
    slli   t3, t1, 2
    add    t3, t0, t3
    lw     t3, 0(t3)
    sub    t2, t2, t3
    j      ciclo

exit:
```

Che valore contiene il registro t2 al termine del programma? Motivare la risposta.

- a) 0
- b) -20
- c) 10
- d) Nessuna delle precedenti

3. (1 pt) Un file oggetto:
- a) Contiene sempre un segmento di testo e uno di dati statici inizializzati
 - b) Contiene sempre un segmento heap e uno stack
 - c) Contiene sempre un header e un segmento testo
 - d) Nessuna delle precedenti

PARTE 2 – (POSSIBILI) RISPOSTE MULTIPLE -

Ogni domanda può avere da una a quattro risposte CORRETTE.

- Ogni risposta esatta viene calcolata: +1
 - Ogni risposta errata viene calcolata: -0.5
 - Una risposta lasciata in bianco viene calcolata: 0
4. Relativamente alla addizione floating point, quali affermazioni sono vere?
- a) Esegue in un numero di cicli molto più alto rispetto alla moltiplicazione floating point
 - b) Esegue in un solo ciclo se c'è dell'hardware dedicato (FPU)
 - c) Richiede uno shift per rendere l'esponente dei due operandi uguale
 - d) Può richiedere una normalizzazione dopo la somma
5. La moltiplicazione in RISC-V:
- a) Se implementata tramite hardware dedicato impiega molti meno cicli rispetto al software
 - b) Esegue sulla ALU
 - c) È tipicamente un'operazione multiciclo
 - d) Nella tipica implementazione hardware può condividere logica con la divisione
6. Relativamente al programma mostrato, quali affermazioni sono vere?

fun:

```
li    t0, 5
li    t1, 1
add   s0, t0, t1
mv    a0, s0
jal   ra, fun2
add   a0, a0, t0
ret
```

fun2:

```
addi  t0, a0, 1
ret
```

- a) **fun** non adempie alla calling convention
- b) **fun2** non adempie alla calling convention
- c) **fun** ritorna 12
- d) Non ritorna correttamente al chiamante di **fun**

PARTE 3 – DOMANDE APERTE

- Una risposta esatta fa acquisire il punteggio positivo riportato a fianco della domanda
- Una risposta errata può causare una penalità che dipende dalla gravità dell'errore
- Una risposta lasciata in bianco viene calcolata: 0

7. (11 pt) Si consideri il seguente frammento di memoria istruzioni (indirizzo, istruzione):

0x44	0x00052483
0x48	0x00940433
0x4C	0x00450513
0x50	0xFEB54AE7

Basandosi sulle tabelle di decodifica fornite in appendice:

- Tradurre le istruzioni in un corrispondente stralcio di programma assembly. [5 pt]
- Dire se nel programma tradotto ci sono istruzioni che alterano il control flow. In caso affermativo, osservando gli indirizzi a cui sono memorizzate le istruzioni tradurre opportunamente i loro campi immediato in modo da riscrivere il programma assembly con le corrispondenti label [2 pt]
- Sapendo che nel programma originale prima dello stralcio di codice ai punti a) e b) si trovano le seguenti istruzioni

li	x8, 0
slli	x11, x11, 2
add	x11, x11, x10

e sapendo che in x10 (a0) si trova l'indirizzo di base di un array A (inferire il tipo di A dalle istruzioni date) e in x11 (a1) il numero N di elementi contenuti nell'array A:

Dire cosa fa il programma (fornire anche il commento linea per linea) [2 pt]

- Il codice ricostruito al punto c) costituisce il corpo di una funzione **func**. Dire se da calling convention è necessario gestire lo stack. In caso affermativo aggiungere al codice le istruzioni opportune. Gestire correttamente il dato ritornato. [2 pt]

8. (8 pt) Si scriva una funzione **serie** che prende in ingresso due interi n ed m e ritorna la sommatoria mostrata sotto. Se n è non positivo la funzione ritorna 0.

$$n^m + (n - 1)^m + (n - 2)^m + \dots + 1^m$$

Per svolgere il calcolo la funzione **serie** deve chiamare una funzione **pow** che prende in ingresso due interi a e b e restituisce a^b .

Appendice: Tabelle di codifica delle istruzioni RISC-V e registri

Format	Instruction	Opcode	Funct3	Funct6/7
R-type	add	0110011	000	0000000
	sub	0110011	000	0100000
	sll	0110011	001	0000000
	xor	0110011	100	0000000
	srl	0110011	101	0000000
	sra	0110011	101	0000000
	or	0110011	110	0000000
	and	0110011	111	0000000
	lr.d	0110011	011	0001000
	sc.d	0110011	011	0001100
Format	Instruction	Opcode	Funct3	Funct6/7
I-type	lb	0000011	000	n.a.
	lh	0000011	001	n.a.
	lw	0000011	010	n.a.
	ld	0000011	011	n.a.
	lbu	0000011	100	n.a.
	lhu	0000011	101	n.a.
	lwu	0000011	110	n.a.
	addi	0010011	000	n.a.
	slli	0010011	001	0000000
	xori	0010011	100	n.a.
	srlr	0010011	101	0000000
	srai	0010011	101	0100000
	ori	0010011	110	n.a.
	andi	0010011	111	n.a.
	jalr	1100111	000	n.a.
Format	Instruction	Opcode	Funct3	Funct6/7
S-type	sb	0100011	000	n.a.
	sh	0100011	001	n.a.
	sw	0100011	010	n.a.
	sd	0100011	111	n.a.
SB-type	beq	1100111	000	n.a.
	bne	1100111	001	n.a.
	blt	1100111	100	n.a.
	bge	1100111	101	n.a.
	bltu	1100111	110	n.a.
	bgeu	1100111	111	n.a.
U-type	lui	0110111	n.a.	n.a.
UJ-type	jal	1101111	n.a.	n.a.

Name (Field Size)	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits	Comments
R-type	funct7	rs2	rs1	funct3	rd	opcode	Arithmetic instruction format
I-type	immediate[11:0]		rs1	funct3	rd	opcode	Loads & immediate arithmetic
S-type	immed[11:5]	rs2	rs1	funct3	immed[4:0]	opcode	Stores
SB-type	immed[12,10:5]	rs2	rs1	funct3	immed[4:1,11]	opcode	Conditional branch format
UJ-type	immediate[20,10:1,11,19:12]				rd	opcode	Unconditional jump format
U-type	immediate[31:12]				rd	opcode	Upper immediate format

Register(s)	Alt.	Description
x0	zero	The zero register, always zero
x1	ra	The return address register, stores where functions should return
x2	sp	The stack pointer, where the stack ends
x5-x7, x28-x31	t0-t6	The temporary registers
x8-x9, x18-x27	s0-s11	The saved registers
x10-x17	a0-a7	The argument registers, a0-a1 are also return value